



Simulador de Entrevistas Técnicas con IA Generativa

Sistemas Inteligentes

9CNL

Cristian Fernando Clavijo Morales - 93257

Kevin Andres Nuñez Castaño - 94125

Andres Felipe Rodriguez Guzman - 46439

BOGOTA D.C

2026

Propuesta del proyectos del equipo	3
Simulador de Entrevistas Técnicas con IA Generativa	3
Problema y Objetivo	3
Tipo de Solución	3
Justificación Técnica	3
Dataset a trabajar	3
Algoritmos y herramientas posibles	4
Arquitectura	5
Flujo Lógico de la Entrevista (Diagrama de Proceso)	5
Flujo lógico de la entrevista	5
Arquitectura general del sistema	6
Proyecto del equipo (ML o IA generativa)	7
Problema y objetivo	7
Modelo ia seleccionado para la implementación	7
Métricas de evaluación	7
Similitud de coseno	7
Score de calidad	8
Conclusiones	8
Referencias	8

Propuesta del proyectos del equipo

Simulador de Entrevistas Técnicas con IA Generativa

Problema y Objetivo

- **Problema:** Existe una desconexión entre el conocimiento teórico de los estudiantes y candidatos del área tecnológica y su desempeño efectivo en entrevistas técnicas reales. Las plataformas actuales suelen centrarse en validar código funcional o pruebas automatizadas, pero dejan de lado dimensiones importantes como la comunicación técnica, el razonamiento verbal, la argumentación y la capacidad de explicar decisiones técnicas.
- **Objetivo:** Desarrollar un simulador inteligente de entrevistas técnicas que permita practicar en un entorno similar a un proceso de selección real. El sistema debe evaluar respuestas en texto o código, calcular un puntaje interpretable y generar retroalimentación personalizada para que el usuario identifique fortalezas, errores y oportunidades de mejora.

Tipo de Solución

El proyecto corresponde a una solución con IA generativa apoyada en evaluación semántica. No se plantea únicamente como un chatbot, sino como un sistema de evaluación que combina un dataset controlado, embeddings con Sentence-BERT, soporte de conocimiento mediante RAG y generación de feedback con Llama 3.2 ejecutado localmente mediante Ollama.

Justificación Técnica

La viabilidad de este proyecto reside en la transición de la programación imperativa a la computación cognitiva. Se justifica técnicamente mediante:

- RAG permite consultar información real del dataset o de una base de conocimiento antes de generar feedback, reduciendo el riesgo de respuestas inventadas.
- Sentence-BERT permite convertir la respuesta del usuario y la respuesta ideal en vectores para comparar su similitud semántica, incluso cuando no usan exactamente las mismas palabras.
- Llama 3.2 se utiliza para interpretar el score obtenido y generar una retroalimentación clara, personalizada y con lenguaje natural.
- Ollama facilita la ejecución local del modelo, disminuyendo costos por uso, dependencia de internet y exposición de información sensible.
- Los guardrails ayudan a mantener la entrevista dentro de un contexto profesional y técnico.

Dataset a trabajar

El dataset se mantiene como una estructura propia en formato JSON. Su propósito es almacenar preguntas técnicas, respuestas de referencia, palabras clave, nivel de dificultad y categoría. Esta decisión permite controlar la calidad de la evaluación y escalar gradualmente el número de preguntas.

Campo	Descripción
id	Identificador único de la pregunta.
question	Pregunta técnica o escenario que debe resolver el usuario.
ideal_answer	Respuesta de referencia usada como estándar de comparación.
keywords	Conceptos clave esperados en una respuesta adecuada.
difficulty	Nivel de dificultad: junior, mid o senior.
category	Área técnica evaluada, por ejemplo frontend, backend o bases de datos.

Problemas detectados en el dataset inicial: tamaño reducido, baja variedad de escenarios, desbalance entre niveles y posible sesgo hacia ciertas tecnologías. Para mitigarlo, se propone iniciar con 5 a 10 preguntas y escalar progresivamente a 10 a 30 registros, balanceando niveles, categorías y tipos de respuesta.

Algoritmos y herramientas posibles

1. Sentence-BERT: generación de embeddings y cálculo de similitud semántica.
2. RAG con base vectorial: recuperación de respuestas ideales o contexto técnico relevante.
3. Llama 3.2: generación de retroalimentación personalizada a partir del score y el contexto técnico.
4. Ollama: ejecución local del modelo seleccionado para pruebas académicas y prototipo funcional.
5. Prompt engineering y guardrails: control del rol del entrevistador, tono del feedback y límites de seguridad.

Arquitectura

Flujo Lógico de la Entrevista (Diagrama de Proceso)

La evaluación del sistema se apoya en tres tecnologías principales. Primero, Sentence-BERT convierte las respuestas en vectores para calcular la similitud semántica entre la respuesta del usuario y la respuesta ideal. Luego, el motor de evaluación genera un puntaje considerando similitud, palabras clave y cobertura de conceptos. Posteriormente, Llama 3.2 interpreta este resultado y genera una retroalimentación personalizada. Finalmente, el uso de RAG permite que el sistema se apoye en información real del dataset o de la base de conocimiento, reduciendo respuestas inventadas y mejorando la precisión del feedback.

Flujo lógico de la entrevista

1. Selección de rol: El usuario elige su nivel para personalizar la entrevista.
2. Carga de pregunta + RAG: El sistema selecciona una pregunta del dataset y puede consultar información real para enriquecer el contexto.
3. Respuesta del usuario: El usuario envía una respuesta en texto o código.
4. Preprocesamiento: La respuesta se limpia y normaliza antes del análisis.
5. Vectorización con Sentence-BERT: La respuesta del usuario y la respuesta ideal se convierten en embeddings.
6. Comparación semántica: Se calcula similitud semántica, coincidencia de palabras clave y cobertura de conceptos.
7. Feedback con Llama 3.2: El modelo interpreta el score y genera retroalimentación personalizada.
8. Output final: El usuario recibe puntaje, nivel estimado y recomendaciones.



Arquitectura general del sistema

Componente	Función
Frontend (UI)	Interfaz para seleccionar nivel, visualizar preguntas, enviar respuestas y consultar resultados.
Backend API	Orquesta la lógica del sistema y conecta los módulos de evaluación.
Base de datos / Dataset	Almacena preguntas, respuestas ideales, palabras clave, nivel y categoría.
Base vectorial	Guarda embeddings para realizar búsqueda y comparación semántica.
Motor de evaluación	Calcula similitud, relevancia, cobertura, coherencia y exactitud.

LLM generativo	Genera feedback natural, personalizado y útil para el usuario.
----------------	--

Proyecto del equipo (ML o IA generativa)

Problema y objetivo

El mercado laboral en el sector tecnológico es altamente competitivo y exige que los candidatos no solo dominen conocimientos técnicos, sino que también sepan comunicarlos de manera clara y estructurada durante entrevistas técnicas. Estas evaluaciones suelen incluir resolución de problemas en tiempo real, preguntas sobre programación, bases de datos y desarrollo de software, lo que requiere práctica específica bajo condiciones similares a un entorno real.

No obstante, muchos estudiantes universitarios del área tecnológica carecen de espacios adecuados para practicar este tipo de entrevistas con retroalimentación inmediata y personalizada. Aunque poseen bases teóricas sólidas, presentan dificultades para responder bajo presión, estructurar sus ideas y demostrar sus competencias de forma efectiva.

Esta problemática afecta principalmente a estudiantes de ingeniería y carreras afines, recién graduados en búsqueda de su primer empleo y personas en transición hacia el sector TI. El principal impacto radica en la pérdida de oportunidades laborales debido a la falta de preparación práctica, lo que genera inseguridad, frustración y retraso en su desarrollo profesional.

Modelo ia seleccionado para la implementación

Para el prototipo se selecciona Llama 3.2 como modelo generativo principal, ejecutado localmente con Ollama. Esta decisión se basa en la viabilidad académica, ausencia de costo por token durante pruebas, control del entorno y menor dependencia de servicios externos. Modelos como GPT-4o o Claude pueden ofrecer alta capacidad de razonamiento, pero implican mayor dependencia de API, internet y costos de uso. Mistral 7B puede considerarse como alternativa ligera o fallback.

Métricas de evaluación

Similitud de coseno

La similitud semántica se calcula comparando el embedding de la respuesta del usuario con el embedding de la respuesta de referencia.

Fórmula: $\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$

Donde A representa el embedding de la respuesta del usuario y B representa el embedding de la respuesta de referencia.

Score de calidad

Métrica	Peso	Fórmula	Descripción
Relevancia	0.25	Palabras clave detectadas / Total de palabras clave	Pertinencia de la respuesta frente a la pregunta.
Coherencia	0.20	1 - (Contradicciones / Total de afirmaciones)	Consistencia lógica del discurso.
Cobertura	0.30	Aspectos abordados / Aspectos esperados	Compleitud de la respuesta técnica.
Exactitud	0.25	Afirmaciones correctas / Total verificable	Precisión técnica de las afirmaciones.

$$\text{Calidad} = (0.25 \times \text{Relevancia}) + (0.20 \times \text{Coherencia}) + (0.30 \times \text{Cobertura}) + (0.25 \times \text{Exactitud})$$

También se recomienda registrar el tiempo de respuesta, el score promedio por sesión y la tasa de alucinación para monitorear desempeño y confiabilidad del sistema.

Conclusiones

Con esta actualización, el documento mantiene su estructura teórica original y se complementa con la información más importante del PDF para orientar la implementación. El proyecto queda documentado como un simulador de entrevistas técnicas que combina dataset propio, RAG, Sentence-BERT, Llama 3.2 y métricas de evaluación interpretables.

La propuesta no busca reemplazar al entrevistador humano, sino ofrecer un entorno de práctica guiada que evalúe comunicación, razonamiento y criterio técnico, dimensiones que las plataformas tradicionales de ejercicios de código no siempre cubren.

Referencias

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

McCarthy, J. (2004). What is Artificial Intelligence? Stanford University.

Meta AI. (2024). Llama 3.2: Open foundation and multimodal models.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP-IJCNLP.